```
---
---
http_interactions:
- request:
    method: post
    uri: https://api.twitter.com/oauth2/token
    body:
      encoding: UTF-8
      string: grant_type=client_credentials
    headers:
      Accept:
      - "*/*"
      User-Agent:
      - TwitterRubyGem/6.1.0
      Authorization:
      - Basic <big string>
      Content-Type:
      - application/x-www-form-urlencoded; charset=UTF-8
      Accept-Encoding:
      - gzip;q=1.0,deflate;q=0.6,identity;q=0.3
  response:
    status:
      code: 200
      message: OK
```

Each cassette file chronicles two HTTP requests to the Twitter API: one for authentication of the API key and one to get the user data. If you delete the files and try again, you'll hopefully notice that the first test run is slow because the HTTP requests are being made. Once the VCR cassette is in place, the test speeds up. (On my machine, the VCR specs went from to 1.6 to about 0.2 seconds.)

Testing for Error Cases

The application design allows multiple ways to simulate errors for testing purposes:

- You can make an API call to the actual service that results in an error and then capture the result using VCR. You can use this approach in an integration or adapter test.
- You can stub a method that's internal to the adapter. For example, you can stub the client method to return a double that simulates an API error. You'd use this in an adapter test.
- You can stub the adapter to return an unexpected value in a client test.